

Sprint 1 – Role C

1. Orchestrator (AWS Step Functions)

- **Text2SQL_Orchestrator:** Created an Express Workflow state machine for low-latency, synchronous execution.
- **Workflow Logic:** Orchestrates the linear flow: RouterAgent → IntentPlanner → SQLGenerator → SandboxExecutor.
- **State Management:** Passes a unified JSON state object between steps, preserving context (User Query, Intent, SQL, Results).
- **Current State:** Configured to trigger "Stub" (Mock) Lambdas to validate the architecture before real agent logic is injected.

2. API Gateway (Public Interface)

- **Text2SQL_API:** Deployed a REST API (Regional) to expose the orchestrator to the outside world.
- **Endpoint:** POST /query
- **Integration:** Direct AWS Service integration using StartSyncExecution (bypassing Lambda proxies for speed).
- **Security:** Configured IAM Role APIGateway_StepFunctions_Access to securely invoke the Step Function.

3. Frontend UI (Streamlit)

- **Analyst Dashboard (app.py):** A lightweight Python web interface for testing the pipeline.
- **Functionality:** Accepts natural language questions, sends payload to API Gateway, and renders the returned SQL and data tables.
- **Debugging:** Includes a "Debug Agent State" expander to visualize the JSON flow between agents.

4. Supporting Infrastructure (Mock Lambdas)

- **Agent_Router_Stub:** Python 3.11 Lambda returning dummy routing decisions.
- **Agent_Planner_Stub:** Python 3.11 Lambda returning dummy intent JSON.
- **Agent_Generator_Stub:** Python 3.11 Lambda returning static SQL (SELECT sum...).
- **Tool_Executor_Stub:** Python 3.11 Lambda returning a mock result set.

Quick Setup Steps:

1. Ensure you have Python installed

2. **Create virtual environment:** `python -m venv venv`
3. **Activate it:** (Windows: `venv\Scripts\activate`, macOS/Linux: `source venv/bin/activate`)
4. **Install dependencies:** `pip install streamlit requests`
5. **Configure API Endpoint:**
 - Open `app.py`
 - Ensure `API_URL` matches your deployed API Gateway URL (e.g., `https://xyz...amazonaws.com/dev/query`)
6. **Run demo:** `streamlit run app.py`

Quick final notes:

The AWS infrastructure is fully "live" but currently disconnected from the real Java/Python agents being built by Roles A & B.

- **The Orchestrator** is running on "Mock Mode" using the stub Lambdas described above.
- **Integration Point:** To switch to production, we simply update the Step Function to point to the real Agent Lambda ARNs instead of the Stubs.
- **Interface Contract:** The system strictly expects the JSON keys: `user_query`, `router_decision`, `intent`, `generated_sql`, and `execution_result`.